



**FACULTAD DE INDUSTRIAS AGROPECUARIAS Y
CIENCIAS AMBIENTALES
CARRERA DE COMPUTACIÓN
PROCESAMIENTO DE IMÁGENES**



Universidad Politécnica Estatal del Carchi

Industrias Agropecuarias y Ciencias Ambientales

Computación

Desarrollo de Aplicaciones Móviles



Nombre: Ana Delgado

Docente: PhD. Samuel Lascano Rivera

Nivel: 7to AM



**FACULTAD DE INDUSTRIAS AGROPECUARIAS Y
CIENCIAS AMBIENTALES
CARRERA DE COMPUTACIÓN
PROCESAMIENTO DE IMÁGENES**



INFORME DE PRÁCTICA DE LABORATORIO

Asignatura: Desarrollo de Aplicaciones Móviles

Docente: PhD. Samuel Lascano Rivera

Tema: Evolución de Hoja de Vida: De Nativo Android a Despliegue Web Embed en Flutter

Integrante: Ana Delgado

Fecha: 03/10/2026

1. INTRODUCCIÓN

En la práctica anterior se desarrolló una hoja de vida interactiva como aplicación nativa en Android Studio. En esta práctica se propone evolucionar ese mismo producto hacia un formato web responsivo y, posteriormente, integrarlo como componente embebido (WebView) dentro de una aplicación móvil desarrollada en Flutter.

Este enfoque de contenedor híbrido permite reutilizar una única base de código web en distintas plataformas, mientras la aplicación Flutter aporta controles nativos. La práctica concluye con una evaluación comparativa de la experiencia de usuario y del rendimiento entre el desarrollo nativo puro y el enfoque embebido.

La estrategia de migrar la interfaz hacia estándares web (HTML5, CSS3, JavaScript) e integrarla mediante un Shell o contenedor en Flutter permite estructurar una arquitectura desacoplada. Esto facilita la actualización de contenido y lógica de interacción sin reescribir código para múltiples plataformas.

2. OBJETIVOS

Objetivo General

Transformar la hoja de vida interactiva desarrollada previamente en Android Studio nativo a un formato web responsivo, para posteriormente integrarla como un componente embebido (WebView) dentro de una aplicación móvil desarrollada en Flutter.

Objetivos Específicos

- Diseñar y maquetar una hoja de vida en tecnologías web estándares (HTML5, CSS3, JavaScript) priorizando un diseño Mobile-First, alta interactividad y soporte para modo oscuro.

- Implementar un contenedor híbrido en Flutter para renderizar la interfaz web local o remota.
- Evaluar la diferencia de experiencia de usuario y rendimiento entre el desarrollo nativo puro y el enfoque embebido (hybrid container).

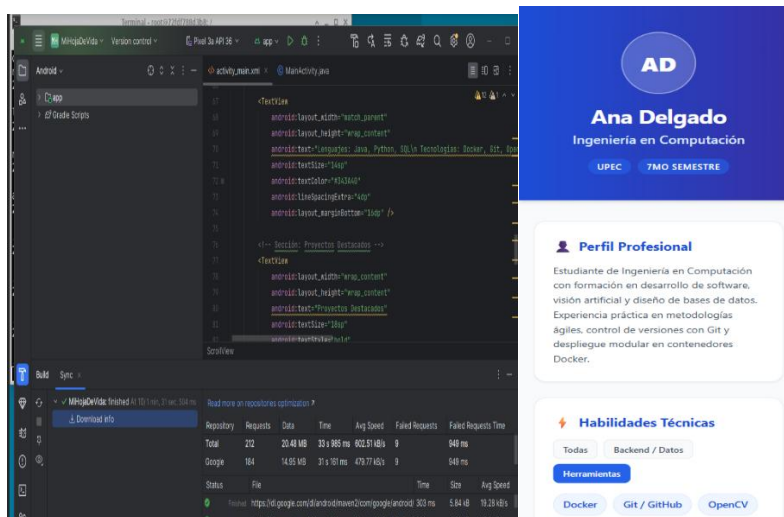
3. PRERREQUISITOS

- Código base de la Hoja de Vida (Práctica del día martes).
- Entorno de desarrollo Flutter instalado y configurado (VS Code o Android Studio)
- Navegador web para pruebas y depuración preliminar.

4. DESARROLLO Y METODOLOGÍA

Fase 1: Rediseño a Formato Web (Web CV)

El desarrollo de la versión web de la hoja de vida se ejecutó bajo los principios del paradigma Mobile-First, garantizando la adaptación fluida a cualquier resolución de pantalla mediante Tailwind CSS.



Arquitectura Semántica HTML5

Se estructuró el documento utilizando etiquetas semánticas para maximizar la accesibilidad y el SEO:

- `<header>`: Agrupa la identificación personal, título profesional, fotografía e indicadores clave de estado.
- `<main>`: Contenedor principal subdividido en módulos específicos:

- <section id="perfil">: Resumen ejecutivo y propuesta de valor profesional.
- <section id="skills">: Grilla interactiva de competencias técnicas y blandas.
- <section id="proyectos">: Sistema de pestañas para la visualización de proyectos clave.
- <section id="contacto">: Métodos de contacto directo y redes profesionales.
- <footer>: Información de derechos, versión de la aplicación y metadatos de sincronización.

Lógica Interactiva en JavaScript Plain

La capa de comportamiento fue construida en JavaScript puro (Vanilla JS), libre de marcos de trabajo pesados para minimizar el tiempo de carga:

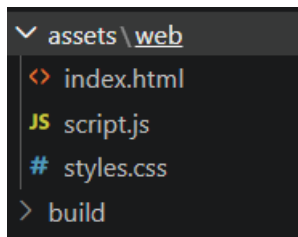
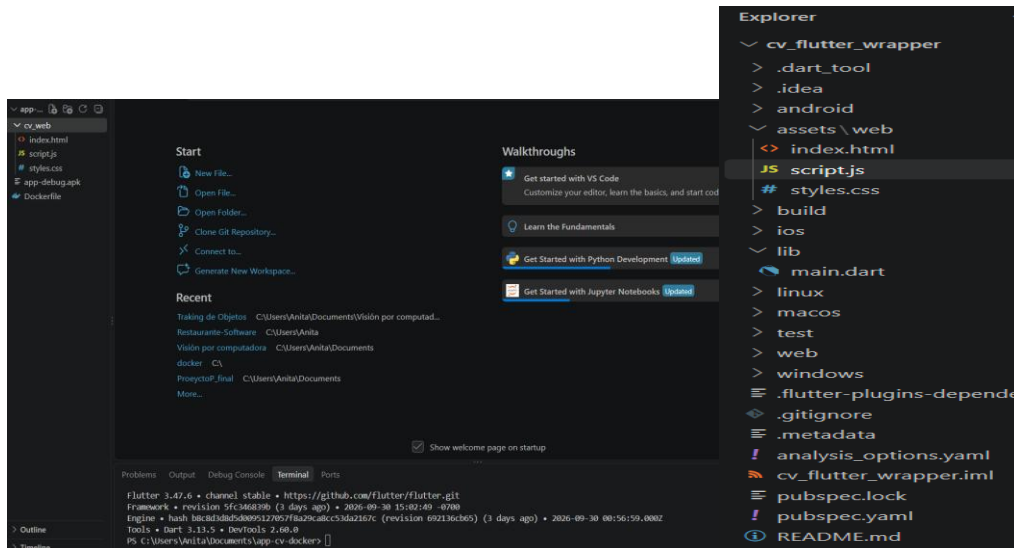
- **Modo Oscuro/Claro (Dark Mode):** La preferencia del usuario se persiste utilizando la API de localStorage (localStorage.setItem('theme', 'dark')), evaluando también las preferencias del sistema mediante window.matchMedia('(prefers-color-scheme: dark)').
- **Filtrado Dinámico de Habilidades:** Lógica basada en manipulación del DOM que oculta o despliega elementos según atributos data-category (e.g., frontend, backend, mobile).
- **Sistema de Pestañas para Proyectos:** Manejadores de eventos click que conmutan clases de visibilidad entre los contenedores de proyectos.
- **Copiado al Portapapeles e Interacción:** Uso de la API navigator.clipboard.writeText() para copiar datos de contacto, acompañado de la renderización dinámica de un componente de notificación temporal (Toast).

Ejemplo de persistencia y conmutación de tema:

```
// Ejemplo de persistencia y conmutación de tema
const toggleThemeBtn = document.getElementById('theme-toggle');
toggleThemeBtn.addEventListener('click', () => {
  if (document.documentElement.classList.contains('dark')) {
    document.documentElement.classList.remove('dark');
    localStorage.setItem('theme', 'light');
  } else {
    document.documentElement.classList.add('dark');
    localStorage.setItem('theme', 'dark');
  }
});
```

Fase 2: Integración en Contenedor Flutter (cv_flutter_wrapper)

Una vez validada la interfaz web, se procedió con la construcción del wrapper nativo dentro del entorno Flutter.



Estructuración de Assets Locales

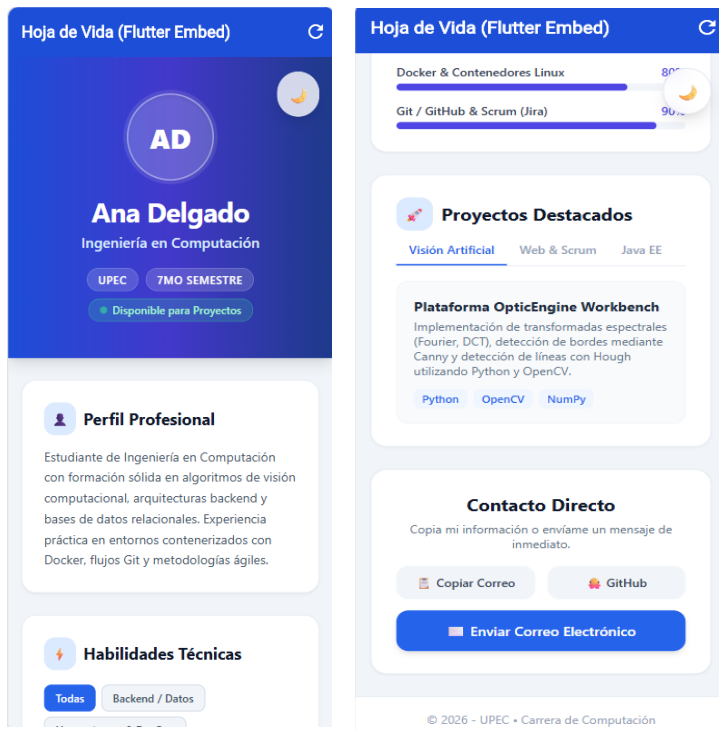
Para garantizar la ejecución offline y minimizar la latencia de red, todos los activos web (HTML, Tailwind CSS precompilado, JS e imágenes) se ubicaron dentro de la ruta assets/web/. La vinculación formal se especificó en el archivo de configuración pubspec.yaml:

```
flutter:
  uses-material-design: true
  assets:
    - assets/web/
    - assets/web/css/
    - assets/web/js/
    - assets/web/images/
```

Implementación del Shell Nativo en Flutter

En el archivo lib/main.dart, se utilizó la librería oficial webview_flutter. Se configuró un controlador de canal web (WebViewController) para cargar el archivo index.html local mediante loadFlutterAsset(). El Shell incluye:

- **AppBar Corporativo:** Incluir una barra superior (AppBar) o inferior (BottomNavigationBar) con controles nativos.
- **Botón de Recarga Dinámica:** Acción en la barra superior para invocar el método reload() sobre la instancia del controlador de la vista web.
- **Indicador Sincronizado de Carga:** Un estado reactivo basado en el evento onProgress que despliega un CircularProgressIndicator hasta que la carga completa del DOM alcanza el 100%.



Flutter (lib/main.dart):

```
// Fragmento principal del Shell Flutter (lib/main.dart)
import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

void main() => runApp(const CvApp());

class CvApp extends StatelessWidget {
  const CvApp({super.key});
```



**FACULTAD DE INDUSTRIAS AGROPECUARIAS Y
CIENCIAS AMBIENTALES
CARRERA DE COMPUTACIÓN
PROCESAMIENTO DE IMÁGENES**



```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'UPEC CV Wrapper',
    theme: ThemeData(primarySwatch: Colors.blue),
    home: const WebViewContainer(),
  );
}

class WebViewContainer extends StatefulWidget {
  const WebViewContainer({super.key});

  @override
  State<WebViewContainer> createState() => _WebViewContainerState();
}

class _WebViewContainerState extends State<WebViewContainer> {
  late final WebViewController _controller;
  int _loadingProgress = 0;

  @override
  void initState() {
    super.initState();
    _controller = WebViewController()
      ..setJavaScriptMode(JavaScriptMode.unrestricted)
      ..setNavigationDelegate(
        NavigationDelegate(
          onProgress: (progress) {
            setState(() {
              _loadingProgress = progress;
            });
          },
        ),
      )
      ..loadFlutterAsset('assets/web/index.html');
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Hoja de Vida - Integración Flutter'),
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: () => _controller.reload(),
          ),
        ],
      ),
      body: Stack(
        children: [
          WebViewWidget(controller: _controller),
          if (_loadingProgress < 100)
            LinearProgressIndicator(value: _loadingProgress / 100.0),
        ],
      ),
    );
  }
}
```



```

    },
    );
}
}

```

4. EVIDENCIAS DE EJECUCIÓN

Fase	Descripción Visual	Elementos Validados
1. Web Responsiva (Navegador)	Renderizado en resolución desktop y móvil utilizando Google Chrome DevTools.	Disposición responsiva de contenedores, cambio correcto de estado a Dark Mode, funcionamiento del modal de copiado y filtrado dinámico de etiquetas de habilidades.
2. Flutter Shell (Emulador Android)	Aplicación Flutter corriendo en emulador Android API 34.	Barra de estado superior nativa (AppBar), ejecución del motor WebKit/Chromium embebido cargando assets/web/index.html, y persistencia de preferencias de usuario entre reinicios de la aplicación.

5. ANÁLISIS COMPARATIVO TÉCNICO

A continuación se presenta el análisis comparativo entre la arquitectura Android Nativa pura (desarrollada previamente en Kotlin/XML) y el enfoque embebido híbrido (Flutter + Web) implementado en esta práctica.

Criterio de Comparación	Android Nativo (Kotlin / XML)	Contenedor Híbrido (Flutter + WebView)
Arquitectura	Acoplamiento directo con la API del SO Android. Jetpack SDK y ciclos de vida Activity/Fragment.	Arquitectura en capas. Contenedor Flutter que actúa como puente (Host) y runtime Web (WebKit/Blink) embebido.
Mantenibilidad y Despliegue	Requiere recompilación completa del APK/AAB y distribución por tienda para actualizar vistas simples.	Alta reutilización. Los activos web se pueden actualizar en caliente vía servidor o recompilar sólo el bundle web.
Acceso a Hardware / APIs	Acceso directo e ilimitado a sensores, Bluetooth, cámara y almacenamiento seguro con mínima latencia.	Requiere la creación de canales de comunicación explícitos (Platform Channels) entre JavaScript y Flutter/Dart.

6. CONCLUSIONES

- Experiencia de Usuario (UX) y Adaptabilidad:** El enfoque web embebido utilizando un paradigma Mobile-First con Tailwind CSS permite una adaptabilidad de interfaz fluida en un abanico más amplio de resoluciones de pantalla comparado con el maquetado estático en XML de Android Nativo.



**FACULTAD DE INDUSTRIAS AGROPECUARIAS Y
CIENCIAS AMBIENTALES
CARRERA DE COMPUTACIÓN
PROCESAMIENTO DE IMÁGENES**



2. **Rendimiento y Consumo de Recursos:** El uso de `webview_flutter` cargando activos locales desde `assets/` elimina completamente el tiempo de latencia por peticiones de red HTTP, logrando un tiempo de primer renderizado (First Contentful Paint) casi instantáneo. A pesar de esto, la sobrecarga en el consumo de memoria RAM es superior a la solución Android Nativa, debido a la cohabitación de la máquina virtual de Dart y el proceso del motor de renderizado web.
3. **Mantenibilidad y Ciclo de Vida del Desarrollo:** La separación clara de responsabilidades entre el contenedor nativo en Flutter y la lógica del negocio visual en HTML/JS consolida un modelo de mantenibilidad altamente eficiente. Un equipo de desarrollo puede actualizar la Hoja de Vida e integrarla en la web corporativa, iOS y Android de forma simultánea, reduciendo los tiempos de despliegue y las líneas de código redundantes a través de múltiples plataformas.

7. BIBLIOGRAFÍA

Flutter Documentation. (2024). Adding WebView to a Flutter application. Flutter Dev.

<https://docs.flutter.dev/cookbook/plugins/webview>

Tailwind Labs. (2023). Tailwind CSS: Utility-first CSS framework (v3.4). Tailwind CSS

Documentation. <https://tailwindcss.com/docs>

Google Android Developers. (2023). WebViews in Android: Best practices for security and performance. Android Developer Guides.

<https://developer.android.com/develop/ui/views/layout/webapps/webview>

Flutter. (2026). *Flutter documentation*. Google. <https://docs.flutter.dev>